

1A

```
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o --lcrypto
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db55c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

1B

```
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8ebb287418aec99c8fc22ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34fb337afedb9b46a1af8e11ec4f476
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$
```

2A

```
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$
```

2B

```
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ xxd .pes/objects/02/1657cde884283370313b3946b7d1253ce80a7893aaa22dc4c2332a
c20bb2d3 | head -n 1
00000000: 7472 656e 2031 339 0031 3030 634 0a   tree 139.10064.
00000010: 5245 4140 4145 2e00 656d 00aa aaaa aaaa README.ed.....
00000020: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa .....
00000030: aaaa aaaa aaaa aaaa aaaa aaaa 108755 .....
00000040: 2062 7569 6c64 2e73 sh00 cccc cccc cccc build.sh.....
00000050: cccc cccc cccc cccc cccc cccc cccc .....
00000060: cccc cccc cccc cccc cccc 3430 3030 .....4000
00000070: 3777 6363 20bbbb bbbb bbbb bbbb bbbb 7wcc .....
00000080: bbbb bbbb bbbb bbbb bbbb bbbb bbbb .....
00000090: bbbb bbbb .....
krish@krish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$
```

3A

```
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ ./pes init
Initialized empty PES repository in .pes/
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ echo "hello" > file1.txt
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ echo "world" > file2.txt
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ ./pes add file1.txt file2.txt
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ ./pes status

Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: commit.h
  untracked: test_objects.c
  untracked: index.c
  untracked: test_sequence.sh
  untracked: pes.h
  untracked: .gitignore
  untracked: test_tree.c
  untracked: index.h
  untracked: test_tree
  untracked: object.c
  untracked: Makefile
  untracked: tree.c
  untracked: commit.c
  untracked: file1.txt
  untracked: file2.txt
  untracked: README.md
  untracked: pes.c
  untracked: test_objects
  untracked: tree.h

krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776316443 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e05d6f606199f7f 1776316443 6 file2.txt
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$
```

3B

4A

```
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ ./pes log
commit df4b6aae7d2d39a84558e5d620996df271ae4cc0ebd5d8530d44abd032c6bcb3
Author: Praj PES1UG24AM141
Date: 1776316755

Add farewell
```

4B

```
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/64/4153d3cfa507af71d06b2d269043f29a5c035706d15336ae1977185d089a5b
.pes/objects/66/224663d23e6f4d99de9e2c7e6d8764305a92d830a1a52d3d5f4aa8007b5c39
.pes/objects/88/c9981c96f17f61b82bea1358011d3349ca032f7ff70701a04e36ca6ce1e76d1
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bbd4d9910c31b4f0e99e4375ffea35c2055130c248ff131f78c6266442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867dd8103f82967099dde814
.pes/objects/df/4b6aae7d2d39a84558e5d620996df271ae4cc0ebd5d8530d44abd032c6bcb3
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

```

krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ cat .pes/refs/heads/main
df4b6aae7d2d39a84558e5d6209996df271ae4cc0ebd5d8530d44abd032c6bcb3
krrish@krrish-VirtualBox:~/Documents/PES1UG24AM141-pes-vcu$ cat .pes/HEAD
ref: refs/heads/main

```

Q5.1: Implementing pes checkout <branch>

To implement a checkout, the system must bridge the gap between the internal object store and the physical files you see in your folder.

- **Internal Changes:** The .pes/HEAD file must be updated to point to the new branch path (e.g., ref: refs/heads/<branch>).
- **Working Directory Changes:** The system must perform a "three-way" sync. It looks at the **Tree** object of the target branch and:
 1. **Creates/Overwrites** files that exist in the target branch.
 2. **Deletes** files that exist in the current directory but are absent in the target branch.
- **The Complexity:** The operation must be **atomic**. If the power cuts out while you are halfway through overwriting main.c, your project is corrupted. Real Git handles this by updating the index first and using temporary files to ensure it can roll back if the operation fails.

Q5.2: Detecting a "Dirty" Working Directory

A "dirty" directory contains work that hasn't been saved to a commit. To detect this conflict without re-hashing every single file (which is slow), you use the **Index** as a cache:

1. **Stat Check:** Compare the current file's size and modification time (mtime) on disk to the metadata stored in the Index. If they differ, the file is modified.
2. **Comparison:** If the file is modified, you check if that specific file is different in the **Target Branch's Tree**.
3. **The Rule:** If the file in your folder is different from your current commit, **and** it's also different in the branch you want to move to, the checkout must fail to prevent your unsaved work from being overwritten.

Q5.3: Detached HEAD

- **The State:** "Detached HEAD" occurs when .pes/HEAD contains a raw **ObjectID** (hash) instead of a reference to a branch file.

- **What happens:** You can still commit! The new commit will point to the previous hash as its parent. However, since no branch (like main) is updated to point to these new commits, they are not part of any "timeline."
 - **Recovery:** If you checkout another branch, you lose the hash of the work you just did. To recover, you would use the reflog (a hidden log of every hash HEAD has ever held) or manually create a branch at that hash: `pes branch new-feature <lost-hash>`.
-

Phase 6: Garbage Collection (GC)

Q6.1: The Mark-and-Sweep Algorithm

Object stores are "append-only," meaning they grow forever. To reclaim space, Git uses a **Mark-and-Sweep** algorithm:

1. **Mark:** Start at the "Roots" (every hash inside refs/heads/*, refs/tags/*, and HEAD). Follow every link: Commits → Trees → Subtrees → Blobs. Every hash reached is marked as "reachable."
 2. **Sweep:** Scan the .pes/objects directory. Any file whose hash was **not** marked is considered "orphaned" and is deleted.
- **Data Structure:** A **Hash Set** (or a highly optimized Bitset) is used to track reachable hashes efficiently.
 - **Estimation:** For 100,000 commits and 50 branches, you might visit roughly **300,000 to 1,000,000 objects**, depending on how much history is shared between branches.

Q6.2: Concurrent GC Race Conditions

- **The Danger:** GC and Commits both touch the object store. A race condition occurs when GC deletes an object that is *currently being created*.
- **The Scenario:** 1. User runs `pes add`. A new Blob is written to .pes/objects. 2. GC starts. It sees this new Blob, but because the user hasn't run `pes commit` yet, nothing "points" to it. 3. GC deletes the Blob as "unreachable." 4. User runs `pes commit`. The commit fails because its intended content was just deleted.
- **The Solution:** Git uses a **grace period** (the prune timer, usually 2 weeks). GC will never delete an object that has a "last modified" timestamp newer than the threshold, giving the user plenty of time to finish their commit.